NumPy Revision Notes

> **Goal:** Learn NumPy well enough for **Python → NumPy → Pandas → Data Engineering**. Don't memorise every function. Understand the concepts below and look up the rest.

How to use these notes

- Read the short explanation, then run (or retype) the code under it.
- Change the numbers and re-run. Seeing what breaks is the fastest way to learn.
- Sections marked ★ are the ones you will use constantly.
- The cheat sheet and learning order are at the end (Part 10).

Table of Contents

Part	Topic	Sections
1	Foundations	What is NumPy, why use it, creating arrays, properties, data types
2	Creating arrays quickly	`zeros`, `ones`, `full`, `arange`, `linspace`, `eye`, random, seed
3	Indexing & slicing	Indexing, 2D indexing, slicing, reverse, ★ boolean indexing, combining conditions
4	Math & statistics	Operations, ufuncs, aggregations, `argmin` / `argmax`, ★ axis
5	Reshaping & restructuring	`reshape`, `flatten`, `ravel`, transpose, join, stack, split
6	Sorting & searching	`sort`, `where`, `unique`
7	Core concepts	★ Copy vs view, ★ broadcasting
8	Matrices & linear algebra	`@`, `dot`, `np.linalg`
9	Missing values & files	`NaN`, `nan*` functions, save/load
10	Cheat sheet & roadmap	Function list, learning order

Part 1 — Foundations

1.1 What is NumPy?

NumPy = Numerical Python. It is a library for:

- Working with numbers and arrays
- Mathematical, statistical and matrix operations
- Handling large amounts of numerical data efficiently

The core object is the **NumPy array (`ndarray`)**. By convention NumPy is imported as `np`:

PYTHON

```python
import numpy as np
```

1.2 Why do we need NumPy?

Python lists are general purpose. NumPy arrays are built for **numerical work**, where operations apply to every element at once (this is called *vectorization*).

PYTHON

```python
numbers = [1, 2, 3, 4, 5]     # Python list
arr = np.array([1, 2, 3, 4, 5])  # NumPy array

print("list  * 2 :", numbers * 2)   # repeats the list
print("array * 2 :", arr * 2)       # multiplies every element
```

OUTPUT

```
list  * 2 : [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
array * 2 : [ 2  4  6  8 10]
```

> **Remember:** `list * 2` repeats the list. `array * 2` does maths on every element.

1.3 Creating arrays with `np.array()`

Think of dimensions like this:

Dimension	Looks like	Example
1D	a row of values	`[1, 2, 3]`
2D	rows and columns (a table)	`[[1, 2, 3], [4, 5, 6]]`
3D	several 2D tables stacked	`[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]`

PYTHON

```python
a1 = np.array([10, 20, 30, 40])            # 1D
a2 = np.array([[1, 2, 3],
               [4, 5, 6]])                 # 2D
a3 = np.array([[[1, 2], [3, 4]],
               [[5, 6], [7, 8]]])          # 3D

print(a1)
print(a2)
print(a3)
```

OUTPUT

```
[10 20 30 40]
[[1 2 3]
 [4 5 6]]
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

1.4 ★ Important array properties

Property	Meaning
`ndim`	number of dimensions
`shape`	size of each dimension, as `(rows, columns)` for 2D
`size`	total number of elements
`dtype`	data type of the elements
`itemsize`	bytes used by each element

PYTHON

```python
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print("ndim     :", arr.ndim)
print("shape    :", arr.shape)    # 2 rows, 3 columns
print("size     :", arr.size)
print("dtype    :", arr.dtype)
print("itemsize :", arr.itemsize)
```

OUTPUT

```
ndim     : 2
shape    : (2, 3)
size     : 6
dtype    : int64
itemsize : 8
```

1.5 Data types

Common types: `int32`, `int64`, `float32`, `float64`, `bool`, `complex`.

Set a type when creating an array with `dtype=`, or convert later with `.astype()`.

PYTHON

```python
f = np.array([1, 2, 3], dtype=np.float64)
print(f)

x = np.array([1.7, 2.2, 3.9])
print(x.astype(int))      # converts by truncating the decimals
print(f.astype(int))
```

OUTPUT

```
[1. 2. 3.]
[1 2 3]
[1 2 3]
```

Part 2 — Creating Arrays Quickly

2.1 `np.zeros()` and `np.ones()`

Arrays filled with `0` or `1`. For 2D, pass the shape as a **tuple**.

PYTHON

```python
print(np.zeros(5))
print(np.zeros((2, 3)))
print(np.ones(5))
print(np.ones((2, 3)))
```

OUTPUT

```
[0. 0. 0. 0. 0.]
[[0. 0. 0.]
 [0. 0. 0.]]
[1. 1. 1. 1. 1.]
[[1. 1. 1.]
 [1. 1. 1.]]
```

2.2 `np.full()`

Fill an array with any value you choose.

PYTHON

```python
print(np.full(5, 10))
print(np.full((2, 3), 7))
```

OUTPUT

```
[10 10 10 10 10]
[[7 7 7]
 [7 7 7]]
```

2.3 `np.arange()`

Like Python's `range()`, but returns an array. Signature: `np.arange(start, stop, step)`.

> **Remember:** `stop` is **not included**.

PYTHON

```python
print(np.arange(1, 10))
print(np.arange(1, 10, 2))
```

OUTPUT

```
[1 2 3 4 5 6 7 8 9]
[1 3 5 7 9]
```

2.4 `np.linspace()`

Creates **evenly spaced** numbers between a start and an end (end **is** included).

Function	You control
`np.arange()`	the **step** between values
`np.linspace()`	the **number of values**

PYTHON

```python
# start=0, end=10, number of values=5
print(np.linspace(0, 10, 5))
```

OUTPUT

```
[ 0.   2.5  5.   7.5 10. ]
```

2.5 Identity matrix: `np.eye()`

1s on the diagonal, 0s elsewhere.

PYTHON

```python
print(np.eye(3))
```

OUTPUT

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

2.6 Random numbers

NumPy has a `np.random` module.

Function	What it gives you
`np.random.rand(n)`	`n` random decimals between 0 and 1
`np.random.randint(low, high, size)`	random integers, `high` is **not included**
`np.random.randn(n)`	`n` values from a standard normal distribution

2.7 Random seed

`np.random.seed(n)` makes random results **reproducible**. Useful for examples, testing and ML.

PYTHON

```python
np.random.seed(42)

print(np.random.rand(5))
print(np.random.randint(1, 10, 5))          # min=1, max=9, 5 values
print(np.random.randint(1, 100, (3, 4)))    # a 3 x 4 array
```

OUTPUT

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
[3 7 8 5 4]
[[24  3 22 53]
 [ 2 88 30 38]
 [ 2 64 60 21]]
```

Tip: `np.random.seed()` is the older style. Newer NumPy code often uses `rng = np.random.default_rng(42)` then `rng.integers(...)`, `rng.random(...)`. Both work. Know the old style because you'll see it everywhere.

Part 3 — Indexing & Slicing

3.1 Indexing (1D)

Works like Python lists, including **negative indexing**.

PYTHON

```python
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])    # first
print(arr[2])
print(arr[-1])   # last
```

OUTPUT

```
10
30
50
```

3.2 Indexing (2D)

Syntax: `arr[row, column]`

PYTHON

```python
arr2 = np.array([[10, 20, 30],
                 [40, 50, 60],
                 [70, 80, 90]])

print(arr2[1, 2])    # row 1, column 2
```

OUTPUT

```
60
```

3.3 Slicing

Syntax: `arr[start:stop]` (stop not included). For 2D: `arr[row_slice, col_slice]`.

PYTHON

```python
print(arr[1:4])

# rows 0 to 1, columns 1 to 2
print(arr2[0:2, 1:3])
```

OUTPUT

```
[20 30 40]
[[20 30]
 [50 60]]
```

3.4 Reversing an array

PYTHON

```python
print(arr[::-1])
print(arr2[::-1, ::-1])   # reverses both rows and columns
```

OUTPUT

```
[50 40 30 20 10]
[[90 80 70]
 [60 50 40]
 [30 20 10]]
```

3.5 ★ Boolean indexing

One of the **most important NumPy concepts** for data analysis. A condition produces a True/False array. Using that as an index keeps only the `True` positions.

PYTHON

```python
arr = np.array([10, 20, 30, 40, 50])

mask = arr > 25
print(mask)
print(arr[mask])        # same as arr[arr > 25]
```

OUTPUT

```
[False False  True  True  True]
[30 40 50]
```

3.6 ★ Combining conditions

Use `&` (and), `|` (or), `~` (not), and **wrap each condition in brackets**.

> **Remember:** For NumPy conditions use `& | ~`, **not** Python's `and or not`.

PYTHON

```python
print(arr[(arr > 10) & (arr < 50)])   # greater than 10 AND less than 50
print(arr[(arr < 20) | (arr > 40)])   # less than 20 OR greater than 40
print(arr[~(arr > 25)])               # NOT greater than 25
```

OUTPUT

```
[20 30 40]
[10 50]
[10 20]
```

Part 4 — Math & Statistics

4.1 Mathematical operations (vectorized)

Operations apply to **every element** with no loop needed.

PYTHON

```python
arr = np.array([1, 2, 3, 4])

print(arr + 10)
print(arr - 2)
print(arr * 2)
print(arr / 2)
print(arr ** 2)
```

OUTPUT

```
[11 12 13 14]
[-1  0  1  2]
[2 4 6 8]
[0.5 1.  1.5 2. ]
[ 1  4  9 16]
```

4.2 Operations between arrays

Arrays of the same shape are combined **element by element**.

PYTHON

```python
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a * b)
```

OUTPUT

```
[11 22 33]
[10 40 90]
```

4.3 Universal functions (ufuncs)

Maths functions that work directly on whole arrays: `np.sqrt`, `np.abs`, `np.square`, `np.exp`, `np.log`, `np.log10`, `np.sin`, `np.cos`

PYTHON

```python
arr = np.array([1, 4, 9])

print(np.sqrt(arr))
print(np.square(arr))
print(np.log10(np.array([1, 10, 100])))
```

OUTPUT

```
[1. 2. 3.]
[ 1 16 81]
[0. 1. 2.]
```

4.4 ★ Aggregation functions

Reduce many values to one. Very important for data analysis.

PYTHON

```python
arr = np.array([10, 20, 30, 40, 50])

print("sum    :", np.sum(arr))
print("min    :", np.min(arr))
print("max    :", np.max(arr))
print("mean   :", np.mean(arr))
print("median :", np.median(arr))
print("std    :", np.std(arr))    # standard deviation
print("var    :", np.var(arr))    # variance
```

OUTPUT

```
sum    : 150
min    : 10
max    : 50
mean   : 30.0
median : 30.0
std    : 14.142135623730951
var    : 200.0
```

4.5 `argmin()` and `argmax()`

These return the **index** of the smallest or largest value, **not the value itself**.

PYTHON

```python
arr = np.array([10, 50, 20, 5, 30])

print(np.argmax(arr))   # 50 is at index 1
print(np.argmin(arr))   # 5 is at index 3
```

OUTPUT

```
1
3
```

4.6 ★ Axis (very important)

`axis` tells NumPy **which direction** to calculate in. You will use it constantly in NumPy and Pandas.

OUTPUT

```
          columns
       axis=1 →
        [ 1  2  3 ]      axis=0 ↓
        [ 4  5  6 ]
```

Axis	Direction	Result
`axis=0`	collapse the **rows** (go down)	one result **per column**
`axis=1`	collapse the **columns** (go across)	one result **per row**

PYTHON

```python
arr = np.array([[1, 2, 3],
                [4, 5, 6]])    # shape (2, 3)

print(np.sum(arr, axis=0))   # 1+4, 2+5, 3+6
print(np.sum(arr, axis=1))   # 1+2+3, 4+5+6
```

OUTPUT

```
[5 7 9]
[ 6 15]
```

Memory trick: The axis you give is the one that **disappears**. Shape `(2, 3)` with `axis=0` leaves `(3,)`. With `axis=1` it leaves `(2,)`.

Part 5 — Reshaping & Restructuring

5.1 Reshaping

`arr.reshape(rows, cols)` changes the shape without changing the data. The **total number of elements must stay the same**.

PYTHON

```python
arr = np.array([1, 2, 3, 4, 5, 6])

print(arr.reshape(2, 3))
print(arr.reshape(3, 2))

try:
    arr.reshape(4, 2)       # 4 x 2 = 8, but we only have 6 elements
except ValueError as e:
    print("Error:", e)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[[1 2]
 [3 4]
 [5 6]]
Error: cannot reshape array of size 6 into shape (4,2)
```

Tip: Use `-1` to let NumPy work out one dimension: `arr.reshape(2, -1)` gives 2 rows and however many columns fit.

5.2 `flatten()` and `ravel()`

Both convert a multi-dimensional array to 1D.

- `flatten()` always returns a **copy**
- `ravel()` returns a **view** when possible (see 7.1)

PYTHON

```python
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(arr.flatten())
print(arr.ravel())
```

OUTPUT

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
```

5.3 Transpose

Swaps rows and columns. Use `arr.T` or `np.transpose(arr)`.

PYTHON

```python
print(arr)
print(arr.T)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

5.4 Joining arrays

`np.concatenate((a, b))` joins arrays end to end. For 2D arrays, `axis` matters.

PYTHON

```python
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate((a, b)))
```

OUTPUT

```
[1 2 3 4 5 6]
```

5.5 Stacking

Function	Does
`np.vstack((a, b))`	stack **vertically** (new rows)
`np.hstack((a, b))`	stack **horizontally** (side by side)
`np.stack((a, b))`	stack along a **new axis**

PYTHON

```python
print(np.vstack((a, b)))
print(np.hstack((a, b)))
print(np.stack((a, b)))
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
```

5.6 Splitting arrays

- `np.split(arr, n)` splits into `n` **equal** parts (errors if it can't)
- `np.array_split(arr, n)` also handles **uneven** splits

PYTHON

```python
arr = np.array([1, 2, 3, 4, 5, 6])

print(np.split(arr, 3))
print(np.array_split(arr, 4))   # uneven is fine here
```

OUTPUT

```
[array([1, 2]), array([3, 4]), array([5, 6])]
[array([1, 2]), array([3, 4]), array([5]), array([6])]
```

Part 6 — Sorting & Searching

6.1 Sorting

For 2D arrays, choose the direction with `axis`.

PYTHON

```python
arr = np.array([5, 2, 8, 1, 3])
print(np.sort(arr))

m = np.array([[3, 1, 2],
              [9, 7, 8]])
print(np.sort(m, axis=0))   # sort each column
print(np.sort(m, axis=1))   # sort each row
```

OUTPUT

```
[1 2 3 5 8]
[[3 1 2]
 [9 7 8]]
[[1 2 3]
 [7 8 9]]
```

6.2 ★ `np.where()`

Two uses:

1. `np.where(condition)` returns the **indexes** where the condition is true
2. `np.where(condition, x, y)` picks `x` where true, otherwise `y`

PYTHON

```python
arr = np.array([10, 20, 30, 40, 50])

print(np.where(arr > 25))
print(np.where(arr > 25, 1, 0))                  # if > 25 then 1 else 0
print(np.where(arr > 25, "high", "low"))
```

OUTPUT

```
(array([2, 3, 4]),)
[0 0 1 1 1]
['low' 'low' 'high' 'high' 'high']
```

6.3 `np.unique()`

Finds unique values. Add `return_counts=True` to also count them.

PYTHON

```python
arr = np.array([1, 2, 2, 3, 3, 3])

print(np.unique(arr))
print(np.unique(arr, return_counts=True))   # (values, counts)
```

OUTPUT

```
[1 2 3]
(array([1, 2, 3]), array([1, 2, 3]))
```

Part 7 — Core Concepts

7.1 ★ Copy vs view

	Code	Behaviour
View	`b = a.view()`	shares the same data as `a`. Changing `b` **can change** `a`
Copy	`b = a.copy()`	independent. Changing `b` does **not** affect `a`

This matters a lot with large datasets. Slices (like `arr[1:3]`) are views too.

PYTHON

```python
a = np.array([1, 2, 3])

v = a.view()
v[0] = 99
print("a after changing view:", a)     # a changed

a = np.array([1, 2, 3])
c = a.copy()
c[0] = 99
print("a after changing copy:", a)     # a unchanged
```

OUTPUT

```
a after changing view: [99  2  3]
a after changing copy: [1 2 3]
```

7.2 ★ Broadcasting

Broadcasting lets NumPy combine arrays of **different but compatible shapes** by stretching the smaller one.

OUTPUT

```
[1, 2, 3] + 10   →   [1, 2, 3] + [10, 10, 10]   →   [11, 12, 13]
```

It works with multi-dimensional arrays too. Here the row `[10, 20, 30]` is applied to every row:

PYTHON

```python
arr = np.array([1, 2, 3])
print(arr + 10)

m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m + np.array([10, 20, 30]))
```

OUTPUT

```
[11 12 13]
[[11 22 33]
 [14 25 36]]
```

> **Rule of thumb:** Compare shapes from the **right**. Each pair of dimensions must be **equal** or one of them must be **1**. Spend time understanding this deeply.

Part 8 — Matrices & Linear Algebra

8.1 Matrix operations

Operator	Meaning
`A * B`	**element-wise** multiplication
`A @ B` or `np.matmul(A, B)`	true **matrix multiplication**

PYTHON

```python
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

print(A * B)           # element-wise
print(A @ B)           # matrix multiplication
print(np.matmul(A, B)) # same as @
```

OUTPUT

```
[[ 5 12]
 [21 32]]
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

8.2 Dot product

`np.dot(A, B)` gives the dot product and, depending on dimensions, matrix multiplication. In modern code, `@` is usually clearer for matrix multiplication.

PYTHON

```python
print(np.dot(A, B))
```

OUTPUT

```
[[19 22]
 [43 50]]
```

8.3 Linear algebra: `np.linalg`

Function	Does
`np.linalg.inv(A)`	inverse
`np.linalg.det(A)`	determinant
`np.linalg.eig(A)`	eigenvalues and eigenvectors
`np.linalg.solve(A, b)`	solve `A x = b`

PYTHON

```python
print("det :", np.linalg.det(A))
print("inv :\n", np.linalg.inv(A))

b = np.array([5, 6])
print("solve A x = b :", np.linalg.solve(A, b))
```

OUTPUT

```
det : -2.0000000000000004
inv :
 [[-2.   1. ]
 [ 1.5 -0.5]]
solve A x = b : [-4.   4.5]
```

Part 9 — Missing Values & Files

9.1 Handling missing values

NumPy uses `NaN` (Not a Number) for missing numerical values. `np.isnan()` checks for them. `.sum()` on the result counts them.

PYTHON

```python
arr = np.array([1, 2, np.nan, 4])

print(np.isnan(arr))
print("NaN count:", np.isnan(arr).sum())
```

OUTPUT

```
[False False  True False]
NaN count: 1
```

9.2 NaN-safe aggregations

A normal `np.mean()` on data containing `NaN` returns `nan`. Use the `nan*` versions to **ignore** NaN: `np.nanmean`, `np.nanmedian`, `np.nansum`, `np.nanmin`, `np.nanmax`, `np.nanstd`

PYTHON

```python
print(np.mean(arr))      # nan
print(np.nanmean(arr))   # ignores the NaN
print(np.nansum(arr))
```

OUTPUT

```
nan
2.3333333333333335
7.0
```

9.3 Saving and loading arrays

Function	Use
`np.save("file.npy", arr)` / `np.load("file.npy")`	one array
`np.savez("file.npz", a=a, b=b)`	several arrays in one file

PYTHON

```python
arr = np.array([1, 2, 3])
np.save("data.npy", arr)

loaded = np.load("data.npy")
print(loaded)

np.savez("data.npz", a=np.array([1, 2]), b=np.array([3, 4]))
z = np.load("data.npz")
print(z["a"], z["b"])
```

OUTPUT

```
[1 2 3]
[1 2] [3 4]
```

9.4 NumPy and text/CSV files

`np.loadtxt()` and `np.genfromtxt()` can read text/CSV-like data. For real-world CSV and data analysis you will normally use **Pandas** instead.

Part 10 — Cheat Sheet & Learning Roadmap

10.1 Function cheat sheet

Category	Functions
Array creation	`np.array` `np.zeros` `np.ones` `np.full` `np.arange` `np.linspace` `np.eye`
Random	`np.random.rand` `np.random.randint` `np.random.randn` `np.random.seed`
Properties	`arr.ndim` `arr.shape` `arr.size` `arr.dtype` `arr.itemsize`
Reshaping	`arr.reshape` `arr.flatten` `arr.ravel` `arr.T` `np.transpose`
Math	`np.sqrt` `np.abs` `np.square` `np.exp` `np.log` `np.log10`
Statistics	`np.sum` `np.mean` `np.median` `np.min` `np.max` `np.std` `np.var`
Searching	`np.where` `np.unique` `np.argmax` `np.argmin`
Joining	`np.concatenate` `np.vstack` `np.hstack` `np.stack`
Splitting	`np.split` `np.array_split`
Sorting	`np.sort`
Missing values	`np.isnan` `np.nanmean` `np.nanmedian` `np.nansum`
Linear algebra	`np.dot` `np.matmul` `np.linalg.inv` `np.linalg.det` `np.linalg.solve`

10.2 What to learn, in order

1. Python lists and tuples
2. NumPy array
3. `ndim` / `shape` / `size` / `dtype`
4. Indexing and slicing
5. Array creation
6. Mathematical operations
7. Boolean indexing
8. Aggregation functions
9. Axis
10. Reshape
11. Transpose
12. Concatenate / stack
13. Sorting / searching
14. Broadcasting
15. Copy vs view
16. NaN handling
17. Random

18. Matrix operations
19. Linear algebra

> **Priority topics** for the Python → NumPy → Pandas → Data Engineering path: arrays, indexing/slicing, shape, axis, boolean indexing, vectorization, broadcasting, reshape, aggregation, and NaN handling.